COMPLAINT COMPASS · SCALING STRATEGY

How we scale: six decisions, clear triggers, firm targets

This is our plan of record. Tags: MEASURED on the live run of 1,500 complaints; PROJECTED from measured timings; ASSUMPTION an input we chose. The scaled system is not yet built; the load test in section 6 is the proof point.

---

1. The answer in 30 seconds

> "Classifying a complaint is the slow part and doesn't touch shared data, so we run as many copies of the classifier as we need. Filing changes the issue list, so it is never copied: each business area gets exactly one filing process, and all areas run in parallel. To grow, we add classifier copies or split a busy area."

---

2. Scalability in plain terms

What scaling means here: handling many more complaints per second without slowing down or getting the counts wrong.

What limits it today

Each complaint goes through two steps:

1. **Classify it.** We ask Jev what the complaint is about. This takes about 0.53 seconds, nearly all of it waiting for Jev to reply.
2. **File it.** We compare it with the list of known issues and file it under the matching one, or create a new issue. This takes about 0.02 seconds for most complaints, longer when it needs a Jev check (35%) or the LLM (about 4%).

Today, one program does both steps, one complaint at a time.

Why we can't just run 10 copies of the whole program

The list of known issues is shared. Suppose two copies each get a complaint about the same brand-new problem at the same moment. Both check the list, both find nothing, and both create a new issue. Now that problem appears twice, and the counts are wrong.

So **only one process may update the issue list at a time.** That's the only hard constraint.

> **The rule in one line:** copy step 1 (classify) as much as you like, because it never changes the issue list. Never copy step 2 (file): it gets exactly one process per area.

How we get around it

1. **Separate the two steps.** Step 1 (classify) never touches the issue list, and it's the slow step. So we run many copies of **step 1 only**, in parallel. Each copy classifies a complaint and passes it on to step 2.
2. **Split the issue list by area.** There isn't really one list. There's one per area of the business (payments, delivery, refunds and so on), and a payments complaint is only ever compared with payments issues. So we give **each area its own single process** that does step 2. Each list has only one process updating it, so there are no duplicates. All the areas run at the same time.

How it grows

- **More complaints:** add more copies of step 1 (never step 2).

- **One area overloaded** (for example payments during an outage): split that area into two, such as by region, each with its own process.
- **A new company:** it gets its own areas and processes. Nothing else changes.
- **Cost:** only about 4% of complaints reach the LLM, so cost rises in a straight line with volume.

The numbers PROJECTED from measured timings

- One copy of step 1 handles about **15 complaints a second**.
- One area process handles about **3.5 complaints a second**, so a business with 7 areas handles about 24 a second.
- All of Indian quick commerce produces about **2 complaints a second** on average, if 2% of orders lead to a complaint ASSUMPTION.

In the rest of this guide, step 1 is called **Stage A** and the per-area process is a **Stage B partition**.

---

3. The strategy: six decisions

	Decision	What we do	Why
1	**Split into two stages**	**Stage A** (identity, Jev gate, embedding) runs as a pool of identical workers. **Stage B** (match, verify, LLM, new issues) runs as one worker per business × area.	0.53 s of a typical 0.55 s is the gate, which needs no shared state. Only Stage B touches the issue list, and only within one area.
2	**Partition key = business : area**	Every classified complaint is queued under a key like `grocery:payments`. One worker owns each key.	A complaint is only compared with issues in its own area (`engine.py`, `_components`), so areas never wait for each other.
3	**Queue: Redis Streams now, Kafka at scale**	Redis Streams in Phase 1. Move to Kafka at 10M complaints a month or 20 businesses, whichever comes first.	Redis is one small service for a team of two; Kafka gives durable, ordered partitions at volume.
4	**Store: Postgres 16 + pgvector**	Replace SQLite. Tables split by business and month. An approximate-nearest-neighbour (HNSW) index per area once it holds 5,000 example vectors. Word-match vectors stored sparse.	Many writers, long history, and fast similarity search in one database we can run anywhere.
5	**Parallel checks, ordered commits**	Inside a partition, Jev verify and LLM calls run in parallel; results are committed in arrival order. If the issue list changed meanwhile, that complaint is re-checked.	Raises a partition above its one-at-a-time rate of ~3.5 a second without breaking order. PROJECTED
6	**Fail soft, never stop intake**	Intake always accepts and queues. LLM trouble: park the ~4% LLM cases, file the rest. Jev trouble: route by similarity alone, label it, re-check later.	96.1% of complaints never need the LLM MEASURED, so an LLM outage never stops filing.

---

4. Capacity plan by phase

Peak load is assumed at 5× the monthly average ASSUMPTION. One Stage A worker handles ~15 complaints a second; one Stage B partition ~3.5 a second one at a time PROJECTED.

Phase	Volume a month	Average / peak per second	Stage A workers	Stage B partitions	Data and queue	Model cost a month
1 One business	Up to 1M	0.4 / 2	2 (redundancy)	7	1 Postgres, Redis Streams	$43
2 Many businesses	10M	3.9 / 19	2–4, autoscaled	~70 (10 businesses)	Postgres + read replica, Kafka	$433
3 Optimise	100M	39 / 193	13–16, autoscaled	~700 (100 businesses)	Postgres split by business; shared GPU embedder	$4,330

Model cost = measured $4.33 per 100,000 × volume; hosting excluded. Stage A workers = peak ÷ 15, rounded up, minimum 2.

5. Scaling rules: when we add capacity

Signal	Trigger	Action
Stage A queue wait	Over 2 s for 1 minute	Add a Stage A worker. Remove one when wait stays under 0.2 s for 10 minutes. Never fewer than 2.
A partition's lag	Over 30 s for 5 minutes, or the worker busy over 70% of the time	Split that area by region (e.g. `grocery:payments:west`).
LLM share of complaints	Above 8% for a day (2× today's 3.9%)	Alert: issue list is stale or a new problem is emerging. Review Curator output and cut-offs.
LLM health	p95 response over 5 s or errors over 5%	Circuit breaker: park LLM cases, switch to the second provider path.
Jev health	Errors over 5% for 1 minute	Degraded mode on; re-check those filings when Jev recovers.
Spend	A business reaches its monthly cap	Hard stop on paid calls for that business (the spend cap already exists: `MAX_SPEND_USD`).

6. Targets we commit to

Target	Commitment	Today
Time to file	95% within 2 s	p90 1.11 s, p99 3.54 s MEASURED
Urgent flag (money lost, safety)	Within 60 s of arrival	Flagged on processing
Intake availability	99.9% of complaints accepted	Single process
Duplicate new issues	Zero from concurrency	One lock today
Model cost	Within 10% of $4.33 per 100,000	$4.33 MEASURED

Build plan: 6 weeks, team of two

1. **Weeks 1–2:** Postgres + pgvector migration; Redis Streams; Stage A and Stage B as separate processes.
2. **Weeks 3–4:** Degraded modes, circuit breaker, running totals for the Top 10, a metrics page (queue wait, partition lag, LLM share, cost).
3. **Week 5:** Load test.
4. **Week 6 onward:** Two-week shadow pilot with one operator.

Load test: pass or fail. Replay complaints at **20 a second for one hour** (10× the average of all Indian quick commerce at our 2% assumption). Pass only if: 95% filed within 2 s; zero duplicate issues; zero lost complaints; model cost within 10% of $4.33 per 100,000. Fail on any one, and we fix before the pilot.

7. Why this design works

The one idea to explain first. The list of known issues changes while complaints are processed. If three customers report a brand-new problem at the same moment and three workers handle them at once, each sees “no match” and each may create its own issue: three duplicates. That is why today’s prototype lets one complaint at a time update the list (a lock in `api.py`, line 38). Our design keeps that one-at-a-time rule only where it is needed: per business area.

Where the time goes MEASURED

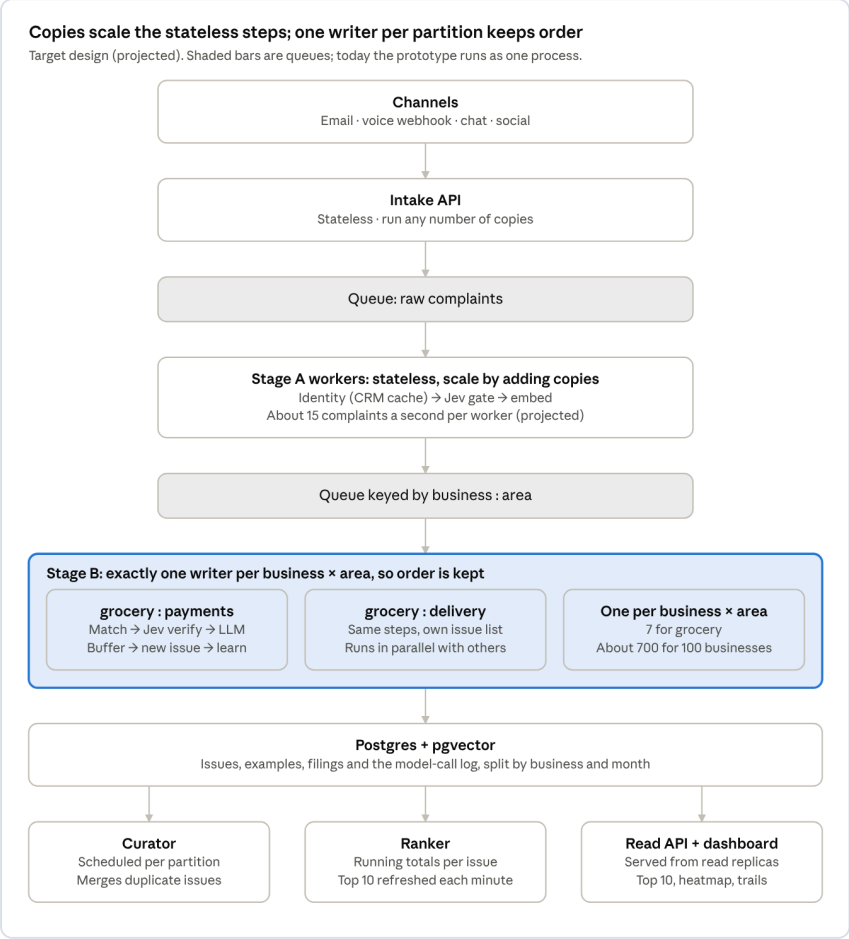
Step	Time	Touches issue list?
Jev gate (all)	0.53 s	No
Similarity match	~0.02 s	Yes
Jev verify (35%)	0.54 s	Yes
LLM decision (~4%)	1.90 s	Yes

HANDOFF.md §4, per-call timings in the live run.

What that means

The slowest step is the one with no shared state, so it scales just by adding workers.

The steps that need order are short for most complaints, and only need order within one area. One area worker averages 0.02 + 35% × 0.54 + 3.9% × 1.90 ≈ 0.28 s per complaint, about 3.5 a second; decision 5 raises that.



Target architecture PROJECTED. Point at this and walk it top to bottom.

8. One complaint through the system

1. An email arrives: “Charged twice for one order after the app update.” Intake accepts it and queues it.
2. A Stage A worker links it to the customer, runs the Jev gate (complaint: yes; area: payments; money lost: yes) and embeds the text. Money lost, so it is flagged urgent.
3. It is queued under `grocery:payments`. The payments worker compares it with known payments issues: a close match is filed free; unsure goes to a Jev yes/no check, then Claude Haiku 5.5 only if still unsure; no match waits until 3 different customers report it, then becomes a new issue.
4. The filing goes to Postgres, the issue’s running totals update, and the Top 10 refreshes within a minute.

9. Failure playbook

Event	What the system does
A payment outage brings 10× complaints	The queue absorbs the burst; most describe the same fault and match free after the first few. If payments lag passes 30 s for 5 minutes, it splits by region.
Claude is slow or down	Circuit breaker parks the ~4% LLM cases and tries the second provider; 96% keep filing.
Jev is down	Degraded mode routes by similarity alone, labels those filings, and re-checks them on recovery.
A new business signs up	Its settings file loads and its partitions start; nobody else is affected. No model retraining.

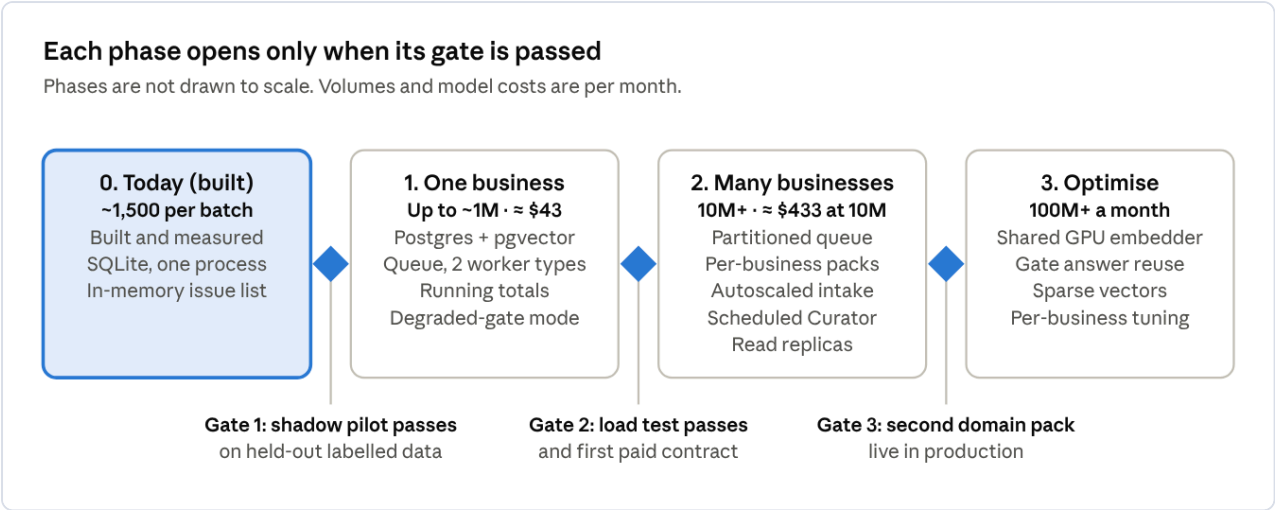
10. Cost at scale, with the exact models

Our funnel uses **Jev 1.14** (a decision model) for the gate and yes/no checks and **Claude Haiku 5.5** for the hard ~4%, via OpenRouter at list prices. The comparison “LLM reads every complaint” is priced, not run, for Claude Haiku 5.5 and Claude Sonnet 5.5.

Complaints a month	Our model cost	Claude Haiku 5.5, best caching	Claude Sonnet 5.5, no caching
1M	$43	$123	$5,940
10M	$433	$1,232	$59,400
100M	$4,330	$12,320	$594,000

$4.33 per 100,000 is our cost, not our price; hosting excluded. List prices per million tokens: Haiku 5.5 $0.10 in / $0.50 out; Sonnet 5.5 $2 / $10.

11. Roadmap and gates



Gate 1 is the shadow pilot; gate 2 is the load test above plus a first paid contract.

Say this before a judge does. The scaled system is designed, not built, and has not been load-tested; throughput figures come from measured timings. Model-provider rate limits at volume are unknown. And scale is not our biggest risk: accuracy on real, messy complaints is, which is why the shadow pilot comes first.

12. Follow-up answers

They ask	You say
"Why not run copies of the whole thing?"	"Copies of the whole program would each create the same new issue. We copy only step 1, classifying, which never touches the issue list; filing gets one process per area."
"What breaks first?"	"The single lock and the SQLite file. Weeks 1–2 replace both: one writer per area, and Postgres."
"When do you add capacity?"	"On fixed triggers: queue wait over 2 seconds adds a worker; a partition lagging 30 seconds for 5 minutes gets split."
"How will you prove it?"	"A one-hour load test at 20 complaints a second. Pass means 95% filed within 2 seconds, zero duplicates, zero lost, cost within 10%."
"Is $4.33 what you charge?"	"No, it's what we pay Jev and Claude per 100,000 complaints. We propose a platform fee plus a charge per complaint."
"What if real data is messier?"	"Cost rises in a straight line. If 20% needed the LLM, it goes from $4.33 to about $6.40 per 100,000."

Close with: "Two stages, one writer per area, fixed triggers for capacity, a 2-second target, and a load test that decides whether we're ready. The economics are already measured."

Sources: backend/runs/grocery_report.json; HANDOFF.md §4; SCALABILITY.md; backend/app/api.py, pipeline/engine.py, pipeline/run.py; the Market and Scale report (RedSeer via GAEE for order volume).